aws COMMUNITY DAY HONG KONG

Testing Python Workflows with Floci

Black-box tests, fault injection and coverage

Alex Au and Sky Ng
(Hong Kong Python User Group)



Who are we?



Alex Au

- Ex-Silicon Valley
- Golden Kubestronaut
- HK Python User Group Host

Sky Ng



- Cloud Engineer
- Ex-AWS Cloud Club Captain

SECTION 01

Can end-to-end tests be faster and cheaper, with useful 100% coverage?

Local End-to-End Test

Reduce cloud deployments	Run the workflow and its services locally without creating any AWS resources.
Speed up Testing	Avoid deploying infrastructure to AWS to shorten the feedback loop.

How much code coverage is enough?

`90%? 95%? 99%? 99.9%?`

Which statement or branch can we leave untested?

100% execution coverage
still needs assertions
on business outcomes.

100%

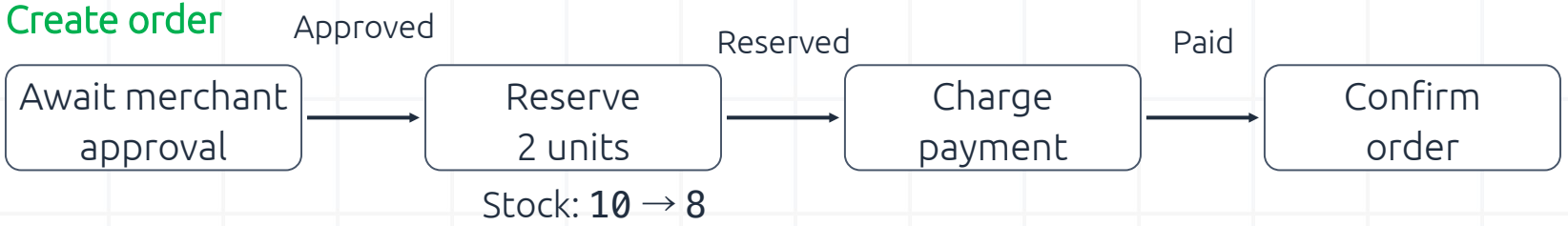
Every statement and branch
in the application's Python code has run.

SECTION 02

What should happen to a purchase order?

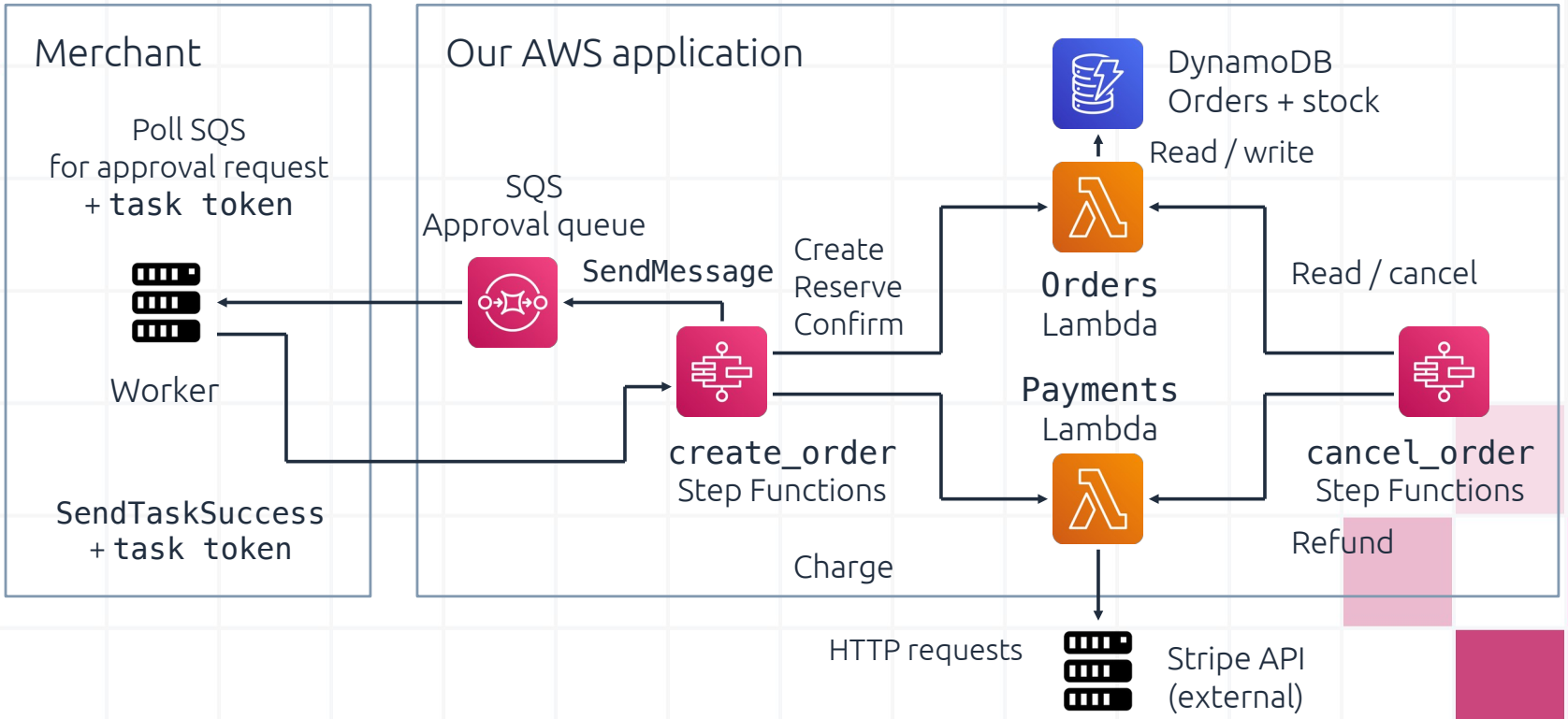
Create and cancel an order (Happy Paths)

Order quantity: 2 units. Initial stock: 10.





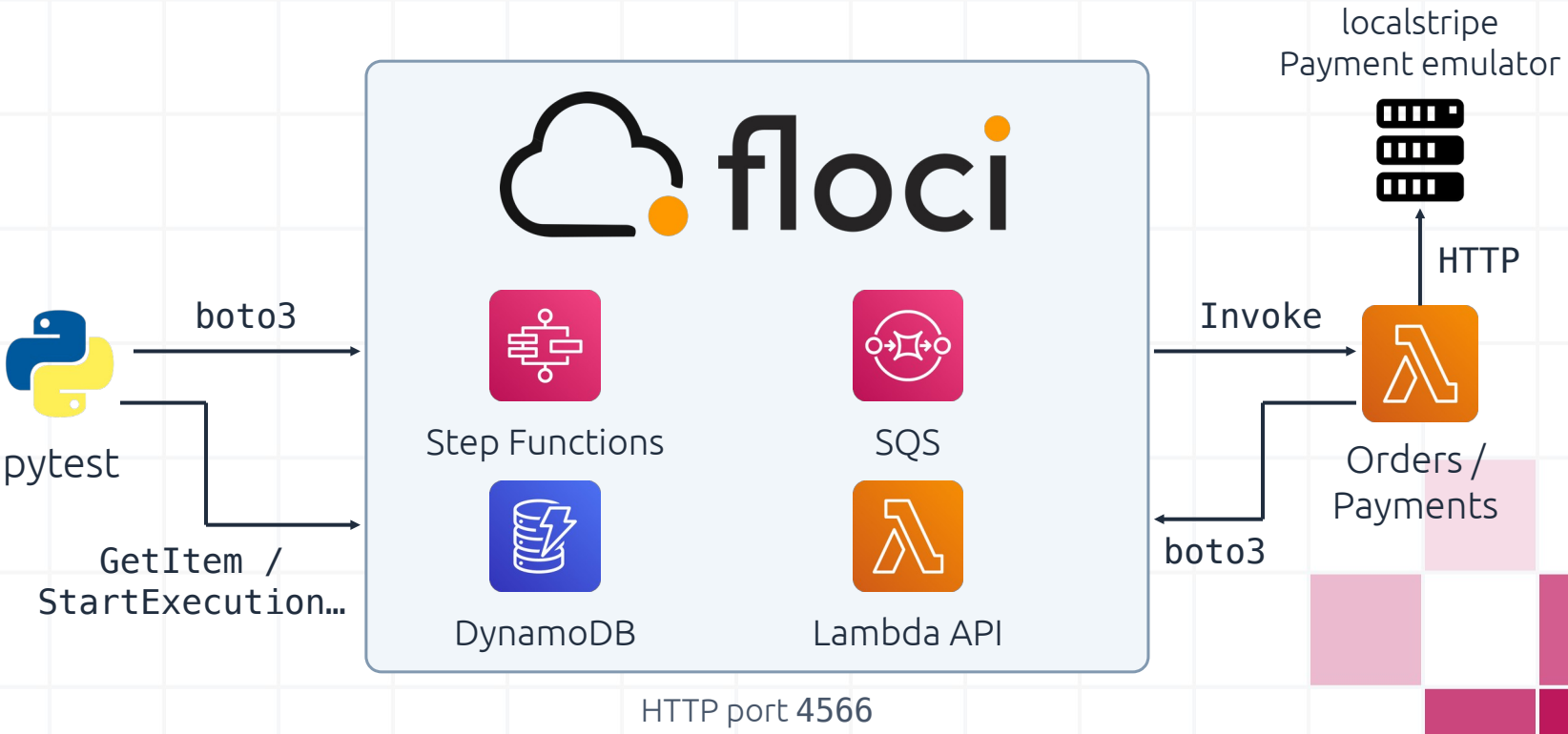
AWS workflow architecture



SECTION 03

How can we properly test these AWS resources locally?

Floci provides local AWS APIs



What a black box test looks like…

test_order_lifecycle.py

```python
execution = sfn_client.start_execution(
    stateMachineArn=create_order_machine,
    input=json.dumps(order_input),
)["executionArn"]
# Wait for merchant approval
# No lookup of Lambda internal state
assert order["status"] == {"S": "CONFIRMED"}
assert stock["Item"]["available"] == {"N": "8"}
assert len(ledger["charges"]) == 1
```

Start the order workflow

Create order

Await merchant approval

Approved

Reserve 2 units

Check new stock availability

Stock: 10 → 8

Charge payment

`ledger`: check localstripe payment records

SECTION 04

What are the roles of **Floci**, **OpenResty** and **coverage.py** in our local e2e test?

Recall our requirements…

Use boto3 SDK and HTTP calls

Specify boto3 `AWS_ENDPOINT_URL` →

floci

Emulate AWS API →

Local End-to-End Test

Reach retry and recovery paths

Specify `HTTP_PROXY` →

OPENRESTY

Inject custom error responses →

Enables testing unhappy paths

Which test ran this Python line?

Set test function name as `COVERAGE_CONTEXT` →



coverage.py

Annotate coverage with test name →

Accountability of test results

Start Floci with a Testcontainers fixture

floci

tests/fixtures/floci.py

```python
@pytest.fixture(scope="session")
def floci_container(
    docker_network: Network, session_id: str
) -> Iterator[DockerContainer]:
    name = f"floci-demo-{session_id}"
    container = (
        DockerContainer("floci/floci:2.3.0@sha256:...")
        .with_name(name)
        .with_network(docker_network)
        # ...
    )
    with container:
        yield container
```

Testcontainers manages
Floci startup and teardown

Get Floci's URL from its container

The Lambda needs Floci's address on the Docker network.

tests/fixtures/floci.py

```python
@pytest.fixture(scope="session")
def floci_internal_url(
    floci_container: DockerContainer,
) -> str:
    container = floci_container.get_wrapped_container()
    return f"http://{container.name}:4566"
```

Use this running Floci container

Put its name in the URL

Connect Lambda to Floci through OpenResty

tests/fixtures/lambdas.py

```
lambda_client.create_function(
    # ...
    Environment={"Variables": {
        "AWS_ENDPOINT_URL": floci_internal_url,
        "HTTP_PROXY": fault_proxy_internal_url,
        "AWS_MAX_ATTEMPTS": "1",
        # ...
    }},
)
```

boto3 sends requests to Floci

OpenResty can inject failures

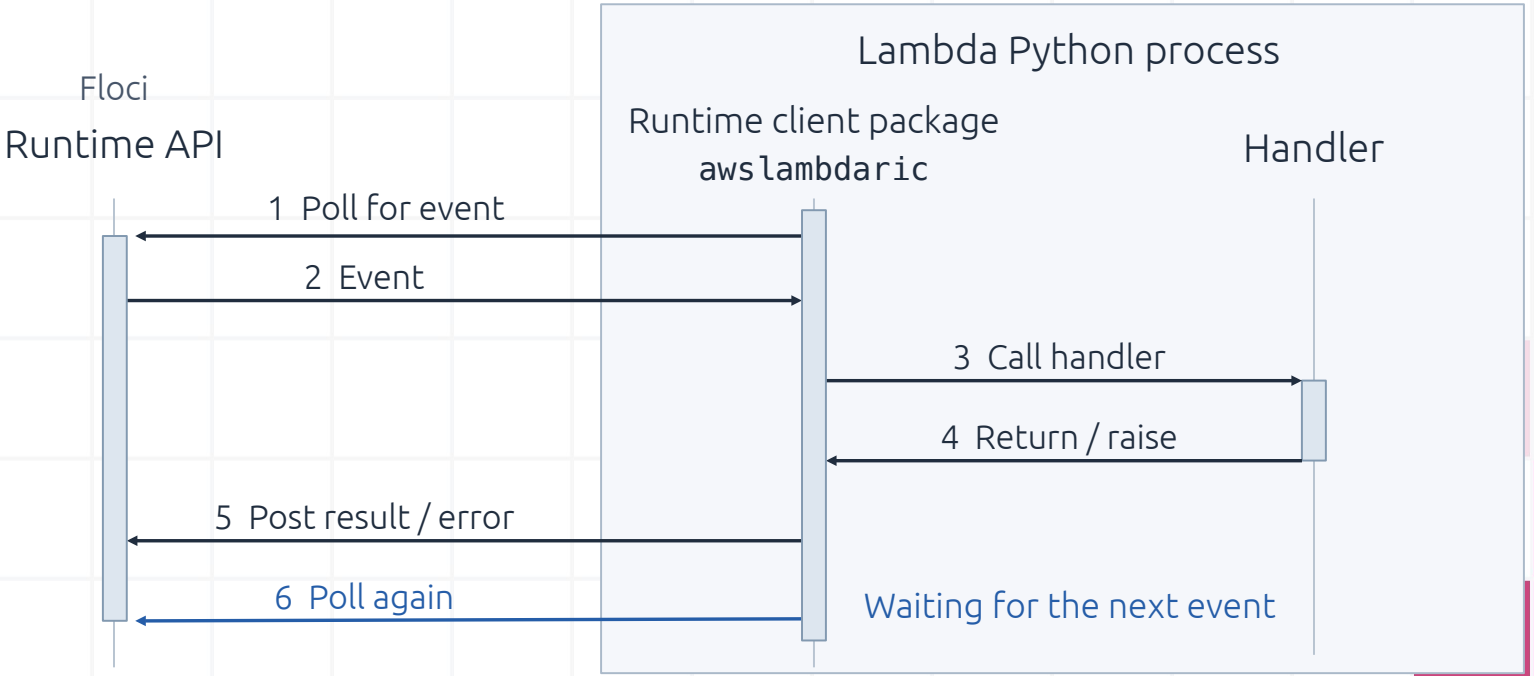
Demo

Floci fixtures and Lambda environment variables

```
open -a Zed \
  tests/fixtures/floci.py \
  tests/fixtures/lambdas.py
```

Not a simple command…

Coverage via pytest cannot trace the Lambda Python in Floci.



Run the Lambda runtime client under coverage.py

Modify AWS Python Lambda image from public ECR

tests/fixtures/lambdas.py

Use coverage.py to start awslambdaric and measure the code it executes.

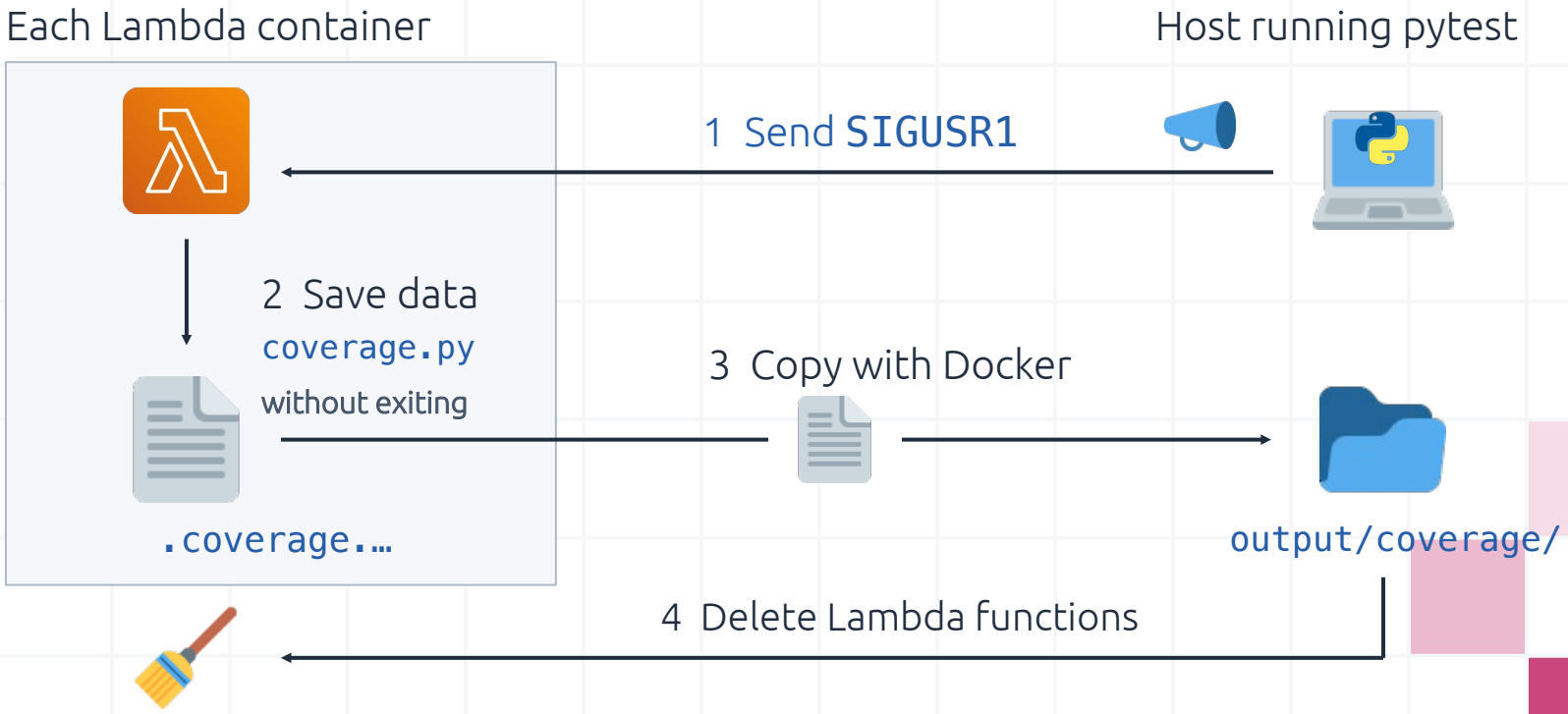
```
ENTRYPOINT ["/var/lang/bin/python3", "-m", "coverage", "run",
"--rcfile=/build/pyproject.toml", "--save-signal=USR1", "-m",
"awslambdaric"]
```

Save coverage without exiting

Load this Lambda handler

```
CMD ["src.lambdas.orders.handler.handle"]
```

Pytest saves coverage before deleting Lambda functions



After all tests: `coverage combine`, then export HTML report.

Show which test executed each line (1)

tests/fixtures/lambdas.py

```
lambda_client.create_function(
    # ...
    Environment={"Variables": {
        # ...
        "COVERAGE_CONTEXT": test_function_name,
        # ...
    }},
)
```

Use test function name to correlate line hit

Show which test executed each line (2)

pyproject.toml

```
[tool.coverage.run]
context = '${COVERAGE_CONTEXT-}'
```

Tag lines with test function name

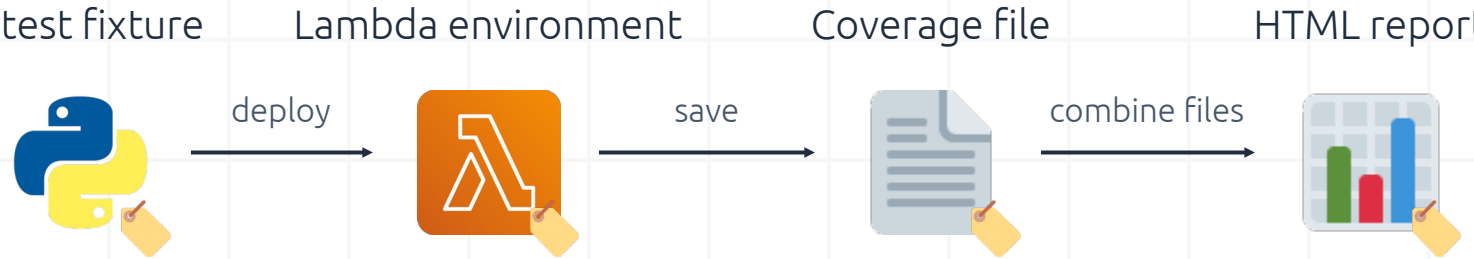
tests/fixtures/lambdas.py

```
ENTRYPOINT ["/var/lang/bin/python3", "-m", "coverage", "run",
"--rcfile=/build/pyproject.toml", "--save-signal=USR1", "-m",
"awslambdaric"]
```

Start awslambdaric with
the coverage context config

The test function name travels with coverage data

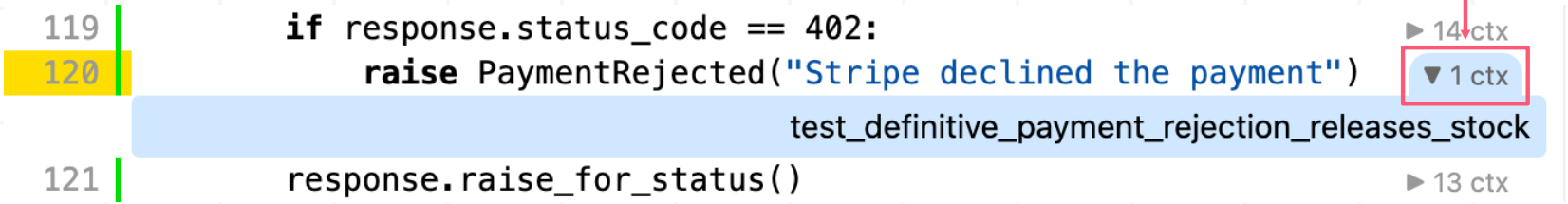
`test_definitive_payment_rejection_releases_stock`



See which test function ran this line

Click “1 ctx” to see the test function name

coverage.py HTML · src/lambdas/payments/client.py



Open HTML report

Demo

Lambda coverage and test function names

```
open -a Zed \
  tests/fixtures/lambdas.py \
  pyproject.toml
```

SECTION 05

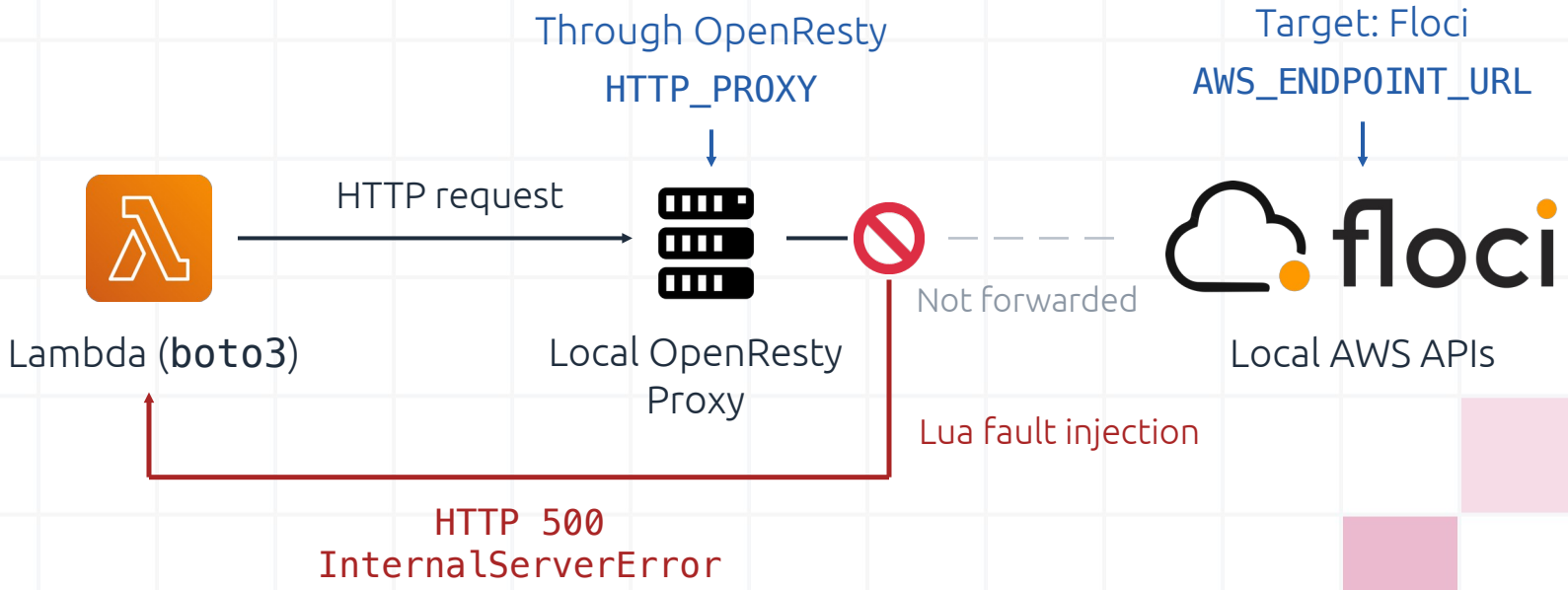
Happy paths are ideal,
how about testing unhappy paths?

The failure point changes the outcome

Situation	Workflow response
Merchant approval deadline expires	`Catch`: cancel the unpaid order
Temporary stock error	`Retry` the stock task
Payment is definitely rejected	`Catch`: restore stock and cancel order
Charge succeeds, reply is lost	Stop and check payment records

How can we test these errors?

OpenResty returns HTTP 500 without calling Floci



Load this test's Lua into OpenResty

Each test mounts its own `/faults/fault.lua`.

NGINX config · tests/fixtures/proxy.py

```
lua_package_path "/faults/?.lua;;";
location / {
    access_by_lua_block {
        require("fault")()
    }
    # ...
    proxy_pass
        http://$http_host$request_uri;
}
```

1 Test writes `fault.lua`

2 Restart OpenResty to reload the Lua module

3 Wait for HTTP 200 from `/health`, then start the workflow

`AWS_MAX_ATTEMPTS=1`: SDK retries off. Step Functions retries the task.

Case 1: reject the first two stock transactions

Lua · fault.lua, written by test_stock_reservation.py

```lua
local remaining = 2
return function()
    if ngx.get_phase() ~= "access" then return end
    local target = ngx.req.get_headers()["x-amz-target"]
    if target == "DynamoDB_20120810.TransactWriteItems"
        and remaining > 0 then
        remaining = remaining - 1
        ngx.status = 500
        ngx.header.content_type = "application/x-amz-json-1.0"
        ngx.say([[{"__type":"InternalServerError",
            "message":"Temporary DynamoDB failure"}]])
        return ngx.exit(500)
    end
end
```

`remaining` = `0`: NGINX forwards requests

Return 500 HTTP Error

Case 2: The charge succeeds before the client times out

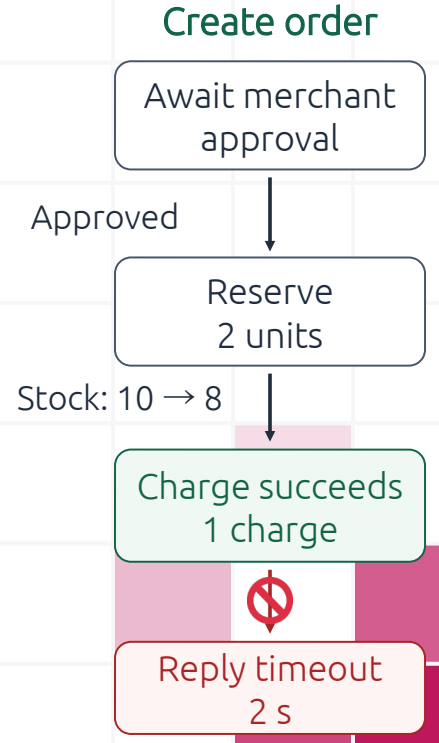
Lua · test_payment_uncertainty.py

1. Charge succeeds
localstripe records it before the delay

```lua
local response = ngx.location.capture(
    "/v1/payment_intents", {
    method = ngx.HTTP_POST,
    body = ngx.req.get_body_data(),
})
ngx.sleep(3.5)
```

2. Client times out after 2 s
OpenResty holds the reply for 3.5 s

Workflow: FAILED · charges: 1
Order: RESERVED · stock: 8

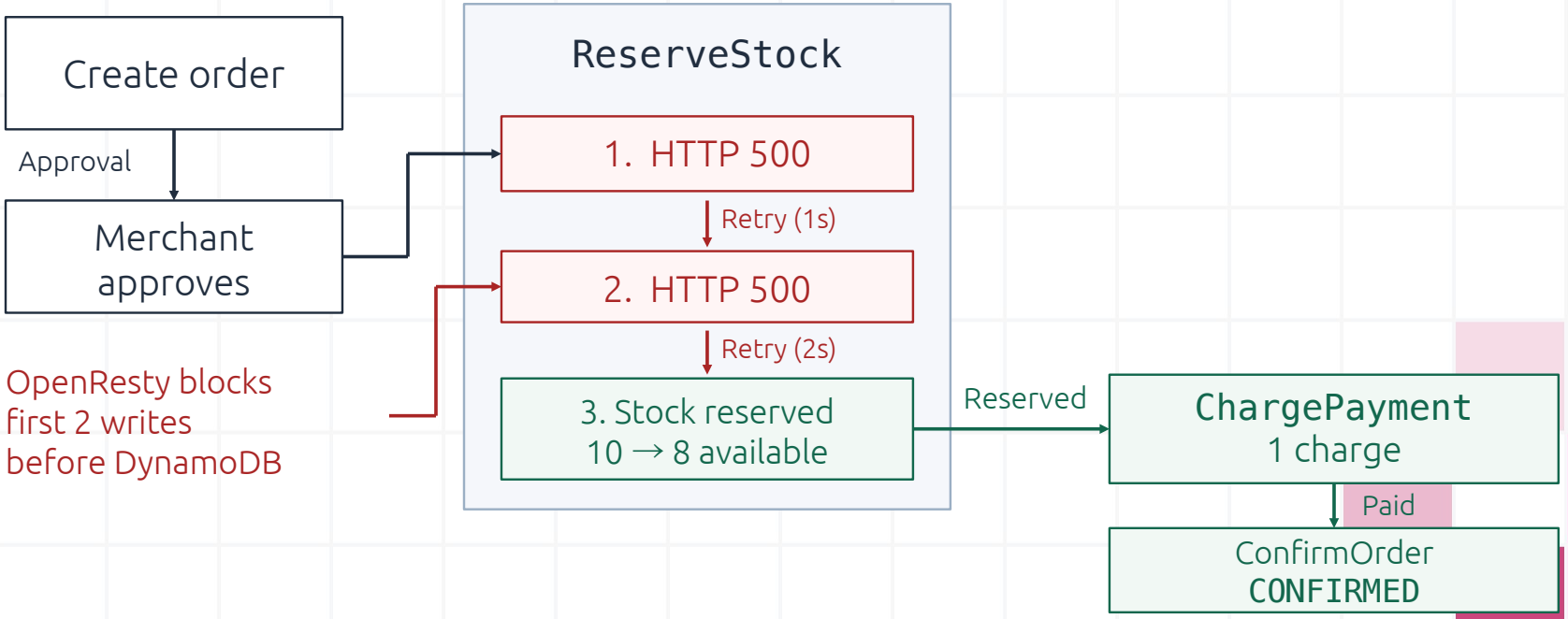


SECTION 06

How do we know the recovery worked?

Stock retry: the results we assert

`create_order` workflow. 2 units. Initial stock: 10.



Demo

Stock retry: one test and its coverage

```
case_name=test_order_transaction_retry_recovers_before_payment
COVERAGE_FAIL_UNDER=0 mise run test -- \
  tests/test_workflows/test_stock_reservation.py::$case_name

open output/htmlcov/index.html
```

Coverage across all 36 tests

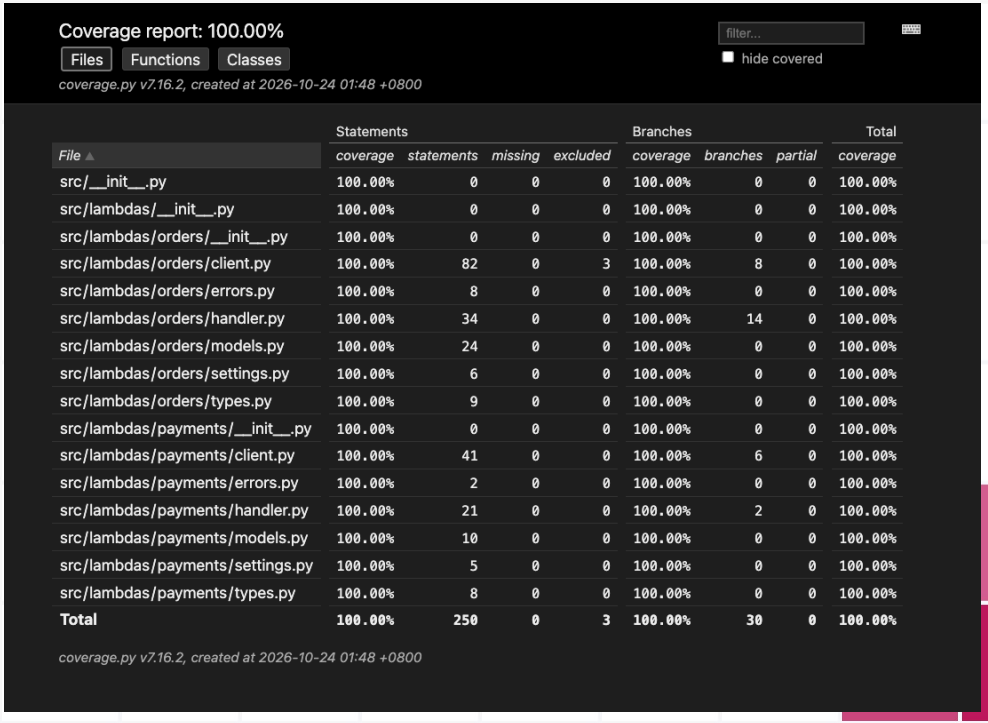
36 tests: workflow E2E + Lambda integrations

250 / 250

Python statement lines executed

30 / 30

Branch outcomes reached



Coverage report: 100.00%

Files | Functions | Classes

filter...

hide covered

coverage.py v7.16.2, created at 2026-10-24 01:48 +0800

	Statements				Branches			Total
File ▲	coverage	statements	missing	excluded	coverage	branches	partial	coverage
src/__init__.py	100.00%	0	0	0	100.00%	0	0	100.00%
src/lambdas/__init__.py	100.00%	0	0	0	100.00%	0	0	100.00%
src/lambdas/orders/__init__.py	100.00%	0	0	0	100.00%	0	0	100.00%
src/lambdas/orders/client.py	100.00%	82	0	3	100.00%	8	0	100.00%
src/lambdas/orders/errors.py	100.00%	8	0	0	100.00%	0	0	100.00%
src/lambdas/orders/handler.py	100.00%	34	0	0	100.00%	14	0	100.00%
src/lambdas/orders/models.py	100.00%	24	0	0	100.00%	0	0	100.00%
src/lambdas/orders/settings.py	100.00%	6	0	0	100.00%	0	0	100.00%
src/lambdas/orders/types.py	100.00%	9	0	0	100.00%	0	0	100.00%
src/lambdas/payments/__init__.py	100.00%	0	0	0	100.00%	0	0	100.00%
src/lambdas/payments/client.py	100.00%	41	0	0	100.00%	6	0	100.00%
src/lambdas/payments/errors.py	100.00%	2	0	0	100.00%	0	0	100.00%
src/lambdas/payments/handler.py	100.00%	21	0	0	100.00%	2	0	100.00%
src/lambdas/payments/models.py	100.00%	10	0	0	100.00%	0	0	100.00%
src/lambdas/payments/settings.py	100.00%	5	0	0	100.00%	0	0	100.00%
src/lambdas/payments/types.py	100.00%	8	0	0	100.00%	0	0	100.00%
Total	**100.00%**	**250**	**0**	**3**	**100.00%**	**30**	**0**	**100.00%**

coverage.py v7.16.2, created at 2026-10-24 01:48 +0800

SECTION 07

What can we claim from these results?

Limits of local testing

Limitation	What to check after local e2e test
localstripe: no `Idempotency-Key` support	Payment retries in Stripe's sandbox
Floci 2.3.0 accepts **expired Step Functions task tokens**	Token expiry on real AWS

100% of which code, exercised by which tests?

Coverage	Did the payment-rejection branch run?
Test context	Which test ran it?
Assertions	Order cancelled, stock restored, and no charge created?

aws COMMUNITY DAY HONG KONG

BONUS

Lambda Durable Functions with MiniStack

The approval-success flow in Step Functions and Python

The states run in the same order; Python replaces the ASL definition.



Step Functions states



Durable Python operations

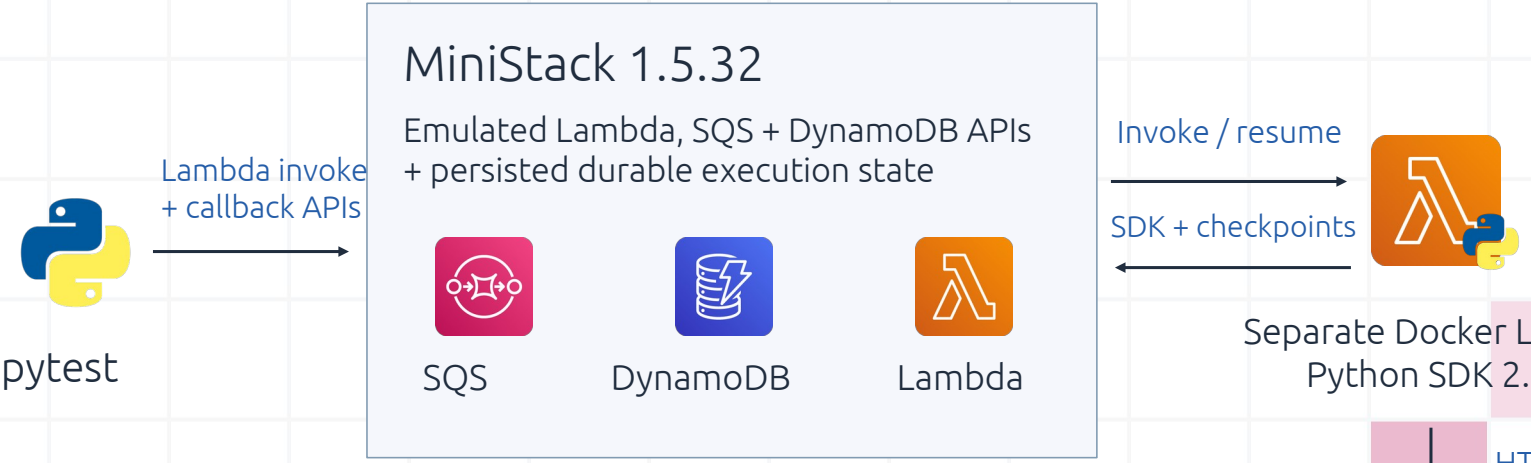
`CreateOrder`	→	`context.step(...)`
`AwaitMerchantApproval`	→	`context.wait_for_callback(...)`
`ReserveStock`	→	`context.step(...)`
`ChargePayment`	→	`context.step(...)`
`ConfirmOrder`	→	`context.step(...)`

Durable = completed step results survive into the next invocation.

Only approval success is implemented here. Retry and compensation are not ported; Lambda coverage is not collected for this bonus.

MiniStack runs this durable workflow locally

MiniStack provides the Durable execution APIs for this local test.
pytest calls those APIs; the workflow runs in a Docker Lambda.



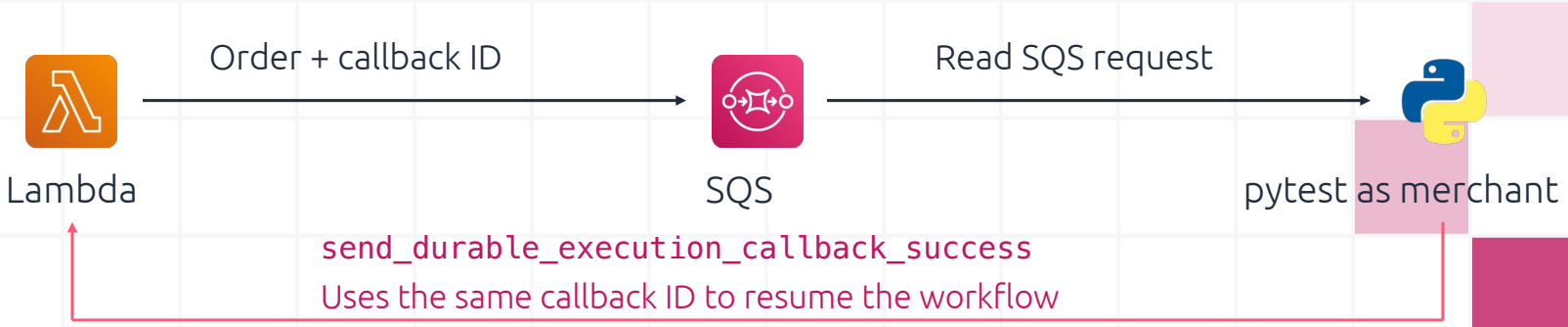
MiniStack serves the local AWS APIs.
localstripe is a separate payment API emulator.

The test answers the approval callback

bonus/durable/workflow.py

```
context.wait_for_callback(
    request_approval,
    name="AwaitMerchantApproval",
    # ...
)
```

Send an approval request, then suspend until its callback is answered.



The callback ID selects the waiting execution

bonus/durable/tests/test_durable_order.py

```
request = json.loads(message["Body"])
callback_id = request["callback_id"]

lambda_client.send_durable_execution_callback_success(
    CallbackId=callback_id, Result=b"{}"
)
```

Read the ID sent through SQS.

Select the waiting task. Success starts invocation 2 of the same execution.

pytest acts as the merchant and sends the reply through the Lambda API.
The ID identifies this execution's wait, so it resumes with its own checkpoints.

Replay returns saved results instead of repeating work

Invocation 1



CreateOrder →

Send approval →



Durable API saves a checkpoint:
order record +
approval-submission completion

Wait for callback

Callback success → invocation 2

Invocation 2

Read checkpoint results





ReserveStock → ChargePayment → ConfirmOrder

The handler starts again,
then reuses saved results.

CreateOrder and the approval request
are not executed again on this replay.

The test checks replay and the stored business result

Durable execution history

```
InvocationCompleted  2
CallbackSucceeded    1

Step                      Started / Succeeded
CreateOrder                     1 / 1
Approval submitter              1 / 1
ReserveStock                    1 / 1
ChargePayment                   1 / 1
ConfirmOrder                    1 / 1
```

Two invocations; the earlier completed steps each start only once.
The test also asserts CONFIRMED, stock 8 and exactly one charge.



Independent DynamoDB reads

Order: CONFIRMED

Stock: 10 → 8



localstripe ledger

Charges: 1

Demo

Approval resumes the workflow; earlier steps run once

```
mise run test-durable
open -a Zed \
  bonus/durable/workflow.py \
  bonus/durable/tests/test_durable_order.py \
  output/durable/merchant-callback.json
```

Q&A



14th Nov – 15th Nov

See you in PyCon HK!